CHƯƠNG 1 · BÀI 2

1.2. Biến, kiểu dữ liệu trong Java

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Nắm các kiểu dữ liệu nguyên thuỷ và kiểu tham chiếu
> - Khai báo biến, hằng số và ép kiểu
> - Sử dụng String đúng cách

1. Kiểu dữ liệu nguyên thuỷ

Java có tám kiểu nguyên thuỷ: `byte`, `short`, `int`, `long`, `float`, `double`, `char` và `boolean`. Kiểu `int` lưu số nguyên 32 bit, `long` lưu số nguyên 64 bit (thêm hậu tố L), `double` là kiểu số thực mặc định, `char` lưu một ký tự Unicode và `boolean` chỉ nhận `true` hoặc `false`.

```
int tuoi = 20;
double diem = 8.5;
char xepLoai = 'A';
boolean dau = true;
long dan = 8_000_000_000L;
```

2. Biến và hằng số

Java là ngôn ngữ định kiểu tĩnh, mỗi biến phải được khai báo kiểu trước khi dùng. Từ khoá `final` dùng để khai báo hằng số, giá trị không thể gán lại: `final double PI = 3.14159;`.

3. Ép kiểu

Chuyển từ kiểu nhỏ sang lớn là tự động (`int` sang `double`); chuyển từ kiểu lớn sang nhỏ phải ép kiểu tường minh và có thể mất dữ liệu.

```
int a = 7;
double b = a;          // tự động: 7.0
int c = (int) 9.8;     // ép kiểu: 9
```

4. Kiểu tham chiếu và String

`String` là kiểu tham chiếu dùng để lưu chuỗi ký tự và là đối tượng bất biến (immutable). Để so sánh nội dung hai chuỗi phải dùng phương thức `equals`, không dùng toán tử `==`, vì `==` chỉ so sánh hai biến có cùng trỏ tới một đối tượng hay không.

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);       // false
System.out.println(a.equals(b));  // true
```

Tóm tắt

- Tám kiểu nguyên thuỷ; `double` là số thực mặc định.
- `final` tạo hằng số; ép kiểu từ lớn xuống nhỏ phải viết tường minh.
- So sánh chuỗi bằng `equals`, không dùng `==`.

Câu hỏi ôn tập và bài tập vận dụng

1. Khai báo biến cho các thông tin của một sinh viên với kiểu dữ liệu phù hợp.
2. Nhập hai số nguyên và in ra thương dưới dạng số thực.
3. Giải thích kết quả của `"abc" == "abc"` và `new String("abc") == new String("abc")`.

Biến, kiểu dữ liệu trong Java — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: đọc một số rồi đọc tên nhưng tên thành chuỗi rỗng. Khi dùng Scanner, nextInt để lại dấu xuống dòng; cách dễ kiểm soát là đọc nguyên dòng rồi chuyển đổi và kiểm tra lỗi.

```
int age = Integer.parseInt("20");
double average = (7 + 8) / 2.0;
System.out.println(age + 1); // 21
System.out.println(average); // 7.5
```

Tự luyện: chuyển `"hai mươi"`, chuỗi rỗng và số quá lớn. **Hướng giải:** parseInt có thể ném NumberFormatException; yêu cầu nhập lại chứ không tự biến dữ liệu sai thành 0. So nội dung String bằng equals hoặc Objects.equals khi cần xử lý null, không dựa vào == để so nội dung.

Nguồn học tiếp

https://dev.java/learn/